A2: Past Performance

Alternative f

Overview

> *Past performance is no guarantee of future results.*

What would have happened if you had given a trading algorithm a sum of money and let it act on historical market data? In **A2: Past Performance**, you will build a *backtester* that answers that question. It reads a file of daily security prices, runs a chosen strategy, and reports the resulting portfolio values.

You do not need prior finance knowledge. The **Glossary (https://canvas.cornell.edu/courses/90271/pages/a2-glossary)** defines the financial terms used here. Please read it before starting the assignment.

Learning Objectives

By the end of this assignment, you should be able to:

- Implement a modular architecture with useful interfaces and hidden representations.
- Express a stateful strategy as a sequence of functional state transitions.
- Build a strategy-independent simulation from smaller, testable components.
- Plan and revise a sequence of working versions of a larger program.
- Design and justify a testing strategy, and make a validation argument using independently checked examples.

Important Updates Made to this Handout

None so far.

The System You Will Build

Your backtester accepts a market file, an output filename, an initial cash amount, and a strategy name. On each day, the backtester reveals that day's closing-prices to the selected strategy. The strategy makes orders, which the backtester processes at that same day's closing prices, and the portfolio is then valued before advancing to the next day. The output file contains the portfolio value for each day.

Alternative f

Software Setup

Follow the Dune project creation instructions on Canvas, including the OUnit setup. Then install these additional packages:

```
opam install batteries csv
```

The `batteries` package provides `BatOptParse`, which you will use for the command-line interface, and the `csv` package provides a CSV parser to handle input and output.

R0. Planning and Testing

A0 and A1 gave you a pre-determined path from the specification to a finished program. A2 gives you more ownership of that path. Each requirement section in this handout begins with explanation and guidance, followed by numbered statements that define **what** your finished submission must do and demonstrate. The requirements do not prescribe the **order** in which you must implement components.

But please be assured: there is no hidden "secret right solution" to discover and no expectation that your first plan or design will survive unchanged. If the additional freedom feels less certain than A0 or A1, use the numbered requirements as the fixed destination and the surrounding guidance to help you choose a route. What matters is the quality of your final product, and how thoughtfully you respond to what you learn.

Read the entire handout and its linked documents so that you understand the assignment. Then, before beginning implementation, create a concise `docs/PROCESS.md` with three or more end-to-end tasks of your own choosing. "End to end" means that each task should describe a complete, observable, and executable behavior of the program – not just an isolated module or function. You should think of these as the prewritten tasks that we gave you in A0 and A1.

Use the provided **Process Template (https://canvas.cornell.edu/courses/90271/pages/a2-process-template)** as a starting point for `docs/PROCESS.md`. Replace its prompts with your own plan and validation record.

Each task should describe **what the executable will do** by the end of that task and identify a **concrete test** you can perform to validate the program at that point. For example, “read a tiny market file, run the cash strategy, and observe one unchanged value per date” would be a good task; but, “implement the `Market` module” is not end-to-end behavior.

As you proceed with implementation, you may discover that your initial plan is incomplete or that a different order of tasks is more effective. You may also find that a task’s test is too weak to give confidence in the program’s correctness. It’s okay! Embrace the iterative nature of development and use these discoveries to improve your approach. In those cases, revise your plan and record the changes in `docs/PROCESS.md`.

We are looking for **honest planning and reflection**, not for a first plan that predicts everything correctly. A0 and A1 gave you very specific instructions. A2 gives you more freedom for deciding how to build and check the program. Treat the plan as a starting hypothesis, keep the program buildable, and revise the plan when implementation teaches you something. A sensible revision is evidence of learning, not a planning failure!

Testing should be part of your development process, not a task to postpone until the end. Use OUnit to check structured OCaml values through interfaces and Cram to check the terminal program output.

R0.1: Submit `docs/PROCESS.md` with at least three end-to-end tasks, planned tests, actual results for completed tasks, and any consequential revisions, as described above.

R0.2: For each completed task, record in `docs/PROCESS.md` a concrete test you performed and its result. A test may be one of the required OUnit scenarios in R7, a partial version of one, or a small command-line run. You do not need a separate, permanent OUnit test for every task.

Suggested progress checkpoints

These are optional guides to pacing, not additional requirements or grading deadlines. The percentages refer to the time you have available for A2. Choose and revise your own end-to-end tasks in `docs/PROCESS.md`; the checkpoints below are merely one possible way to gauge progress.

About 20% through — finish a cash-only run. Read a tiny valid market file, pass its rows through the engine and cash strategy, and check that the program produces one dated value per row, always equal to the starting cash. Write your initial tasks and planned tests in `docs/PROCESS.md` before the implementation grows.

About 40% through — finish portfolio accounting and buy-and-hold. Work out a small multi-date example by hand. Check cash, holdings, and value after each date, including a rejected order

followed by a valid order. Run buy-and-hold through the same engine used for cash.

About 60% through — finish the three strategies behind one interface. Cash, buy-and-hold, and your stateful strategy should all work through the shared strategy signature. Predict the stateful strategy's choices on a tiny market file before writing the test. The engine should not need a branch that implements a particular strategy's rule.

About 80% through — finish the command-line path and required tests. Run each strategy from the command line, inspect its complete summary and `date,value` CSV, and check representative command-line errors. Complete the OUnit and Cram groups described in R7 using expected results calculated independently of the program.

About 90% through — finish the validation record. In `docs/PROCESS.md`, explain the expected results, what your new market files exercise, the tests you ran and their results, and any consequential revisions. Compare the written predictions with the actual dated states and complete command-line reports; investigate mismatches rather than merely relying on a green test result.

By 100% — finish submission checks. Try a new valid market file with different security names, run `dune build` and `dune test` from the project root, review every item in the final checklist, and verify that the submitted files include the market files used by your tests and the completed `docs/PROCESS.md`.

R1. Market Data

Your program reads an input CSV of market data with one date column followed by at least one security column. We call this input a **market file**. The security columns contain closing prices for each security on each date. Here is an example with two securities and three dates:

```
date,ALFA,BETA
2024-01-02,10.00,25.00
2024-01-03,12.00,20.00
2024-01-04,11.00,21.00
```

Download **a2-data.zip (https://canvas.cornell.edu/courses/90271/files/16180859?wrap=1)** **(https://canvas.cornell.edu/courses/90271/files/16180859/download?download_frd=1)** and extract it at the root of your assignment project. This creates the `data/` directory, including `data/handout.csv`, which contains exactly this example.

The exact format is:

- The header is `date` followed by at least one security name. Security names never contain white space or commas. Usually they are a short series of letters. Every security name is unique.

- Every later row has exactly the same number of fields as the header. Blank lines or missing fields would mean the input file is invalid.
- A date has the exact form `YYYY-MM-DD`, is a real Gregorian calendar date, and is later than the date in the preceding row. It might not be the next calendar day, because securities are not necessarily traded every day.
- A price has one or more digits before the decimal point and exactly two digits afterward. It must be greater than `0.00`.
- Fields contain no surrounding whitespace and are not surrounded by quotes.
- A valid file contains a header and at least one data row.

The easiest way to read a CSV file is `Csv.load : string -> Csv.t`, which reads the entire file. The package exposes `Csv.t` as the type `string list list`. The outer list contains rows in file order, and each inner list contains that row's fields in column order.

The `data/` directory in `a2-data.zip` also contains a five-day SPY price dataset; a one-year, five-stock technology dataset; and a large cross-sector dataset spanning the 1999–2001 dot-com collapse. They are based on the public **HSMD dataset (https://www.kaggle.com/datasets/borismarjanovic/price-volume-data-for-all-us-stocks-etfs)**. In R7 below we discuss how to create new market files from the original HSMD data.

You may assume that every market file supplied with this assignment or used for grading has valid contents according to the rules above. You do not need to detect or report violations of those rules.

R1.1: Parse valid input files using the OCaml `csv` package.

R1.2: If the input file cannot be opened (probably because the filename is incorrect, hence it does not exist), produce a useful error message and an unsuccessful exit status rather than exposing an uncaught exception.

R2. Portfolio and Simulation

Note: Make sure you have read the **Glossary (https://canvas.cornell.edu/courses/90271/pages/a2-glossary)** before reading this section. It defines the financial terms used here.

The account begins with a positive amount of cash and no shares. For each date, the backtester engine first reveals that date's closing-prices to the strategy. The strategy then proposes orders, and accepted orders are executed immediately by the engine at those same closing prices.

This is a **simplified end-of-day model**; it does not model intraday trading or the real-world timing required to submit an order before the market closes.

The strategy must never be able to inspect prices from a later row – that is, from a future date. If a strategy needs earlier prices, it must retain them in its private state from earlier calls; the engine does not supply earlier rows again.

Orders are processed in the sequence proposed by the strategy. An order names a security, a buy or sell direction, and a whole-share quantity. The engine must reject an individual order if its quantity is zero or negative, it names an unknown security, it would make cash negative, or it would make a holding negative. A rejection leaves the portfolio unchanged and does not prevent later orders in the sequence from being considered.

Final portfolio value is final cash plus each holding valued at the last listed price. Total return is

```
(final portfolio value - initial cash) / initial cash
```

expressed as a percentage.

Study the **Accounting Reference (https://canvas.cornell.edu/courses/90271/pages/a2-accounting-reference)** to see a worked example of the accounting rules in action.

Represent cash, prices, trade amounts, and portfolio values as integer cents. For example, $1.00 is `100 : int`. Use integer arithmetic for portfolio accounting.

You may use floating-point arithmetic to calculate and format total return. Convert the amounts to floating point before dividing, so that integer division does not discard the fractional part. Multiply the resulting ratio by 100 to express it as a percentage; for example, a ratio of `0.06` means `6.00%`.

You may assume that monetary values and intermediate portfolio accounting calculations in the grading cases fit within OCaml's `int` range. You do not need to detect integer overflow.

All holdings are whole shares. We do not permit fractional shares in this assignment.

R2.1: Process dates in order and calculate each portfolio transition using the rules above.

R2.2: After processing a date's orders, calculate and record portfolio value using that same date's prices.

R2.3: After processing all dates, report the final cash, final holdings, final portfolio value, and total return for the selected strategy. R6 specifies the reporting format.

R3. Strategies

Your program must provide at least three strategies:

- **Cash:** make no orders.
- **Buy and hold:** on the first date, buy as many whole shares as possible of a security named by a command-line option (see R5 for the details on that); make no further orders.
- **Stateful strategy:** use the default momentum strategy below, or replace it with your own more ambitious strategy satisfying R3.1–R3.2. (Or provide both.) In any case, the strategy must choose among the available securities based on historical prices and maintain meaningful private state across dates.

Default stateful strategy: one-day momentum

The default strategy is named `momentum`. It selects a target using the largest strictly positive one-day price increase. It is stateful because it remembers the previous day's prices and the previously selected security. This strategy is deliberately simple: the point is explicit state and a predictable transition, not investment quality.

In more detail, the strategy operates as follows:

- On the first date, remember the current price row, set the target to "none," and make no orders.
- On each later date, subtract each security's preceding-date price from its current price. Select the target security with the largest strictly positive increase. Break ties by choosing the security appearing first in the market file. If no increase is positive, make no orders and retain the preceding date's target.
    - If the selected security differs from the preceding date's target (including when that target is "none"), first sell every share currently held. Then use all available cash, including those sale proceeds, to buy as many whole shares as possible of the selected security. If no shares are affordable, make no buy order but still remember the selected security as the new target.
    - If the selected security is the same as the preceding date's target, make no orders, even if no shares were purchased when that target was first selected.
- After deciding, remember the current price row and the retained or newly selected target for the next date.

Optional original strategy

You are welcome to replace the default `momentum` strategy with your own more sophisticated stateful algorithm. The rule must be deterministic, such that the same market data and initial cash always produce the same orders. Investment return is not graded, but we encourage you to have fun and attempt to maximize your return. Can you create a strategy that performs better than the default? Can you beat the market and make money?! We look forward to seeing what you come up with!

If you use an original strategy, state its rule in `docs/PROCESS.md` so a reader can predict its decisions.

R3.1: At least three strategies must be available: cash, buy-and-hold, and a stateful strategy. The stateful strategy may be the default momentum rule or an original rule.

R3.2: The stateful strategy must consider earlier prices when choosing orders; its retained state must potentially affect later decisions; and changes in the market data must be capable of causing it to select a different security. (The `momentum` strategy satisfies these requirements.)

R4. Architecture and Encapsulation

Before implementing the main components, read the **Architecture Guide (https://canvas.cornell.edu/courses/90271/pages/a2-architecture)** . It provides a recommended default division of responsibilities, explains how information flows between the units, and sketches a suitable shared strategy interface. You may use that overall architecture and interface shape exactly, or you may adapt it to your own design subject to the requirements below.

Each strategy must be implemented as a separate module. Each of those must be sealed with a common module type. In other words, each strategy module implements a common signature. The engine must use that same module type for interacting with all three strategies. It must not call a strategy-specific helper that is not part of the common signature.

In that signature:

- The strategy's **state** must be abstract. The engine should be able to carry that state value from one date to the next without knowing its representation. Clients must not be able to construct or modify a strategy state directly.
- There must be an **update** operation that the engine uses to provide the next day's market data then receive the strategy's proposed orders. In the architecture guide, this is the `step` function. Its input type must accept just one day's observation rather than the complete market data; likewise, it must not provide an operation for retrieving other rows from the engine. The engine is separately responsible for supplying the observation from the date it is currently processing.

R4.1: The design must use a common strategy signature as described above, with an abstract state type and an update operation whose input exposes only one day's data. It must not expose the complete market data or a way to retrieve other rows. Each strategy module must be sealed at that signature, and the engine must use that signature to interact with all strategies.

R4.2: On each simulation date, the engine must pass the update operation the price row for that date and no other past or future row.

The rest of the design is an opportunity for you to practice good use of modularity and encapsulation. Each module should have a clear purpose, and its interface should expose only what is necessary for other modules to use it. You will not, however, be graded on this aspect of your design.

Note: Functors and first-class modules are permitted but not required. Ordinary modules and module types are enough to satisfy the requirements.

R5. Command-Line Interface

Before implementing the command-line interface, read the **Command-Line Option Parsing Guide (https://canvas.cornell.edu/courses/90271/pages/a2-command-line-parsing)** . It gives a complete example of registering string-valued options, generating help, and reporting errors.

Use this command-line shape:

```
dune exec bin/main.exe -- MARKET.csv STRATEGY EQUITY.csv --cash DOLLARS [--hold SYMBOL]
```

Uppercase names are placeholders. `MARKET.csv` is a market file; `STRATEGY` is exactly one strategy name; `EQUITY.csv` is the output CSV of dated portfolio values to write; and `DOLLARS` is a positive amount with exactly two digits after the decimal point, such as `10000.01`. The required strategy names are `cash`, `buy-and-hold`, and `momentum`, unless you replace momentum with an original stateful strategy.

Strategy `buy-and-hold` requires `--hold SYMBOL`, where `SYMBOL` exactly matches a security name in the market file. The other default strategies need no strategy-specific option. You may register both `--cash` and `--hold` for every invocation; reject a missing `--hold` only when buy-and-hold is selected.

For example:

```
dune exec bin/main.exe -- data/medium-tech-2007.csv \
  buy-and-hold equity.csv --cash 10000.00 --hold AAPL
```

This runs buy-and-hold, starting with $10,000.00 and buying AAPL.

R5.1: Run the one selected strategy on the given market file and initial cash and any strategy-specific option.

R5.2: If the command line violates the rules above—for example, it has the wrong number of positional arguments, an unknown strategy name, an invalid cash amount, or a missing or invalid `--hold` value for buy-and-hold—produce a helpful message and an **unsuccessful** exit status without exposing an uncaught exception.

R6. Reports and Output File

Each successfully completed backtest writes the equity CSV named on the command line and prints one complete human-readable final summary to standard output.

In the **equity CSV**, the header is exactly `date,value`. After that, the file contains one data row for each date in the market file, in date order. The first column in each row is the date, and the second column is the portfolio value after processing that date's orders, formatted as a nonnegative dollar amount with exactly two digits after the decimal point. Do not include a dollar sign or thousands separators.

For example:

```
date,value
2024-01-02,100.00
```

After producing equity CSVs, you can visualize one run or compare several strategies by following the **Equity Plotting Guide (https://canvas.cornell.edu/courses/90271/pages/a2-equity-plotting)**. The guide explains how to run the strategies with comparable inputs and use the optional course plotting tool. Creating a plot, however, is not required.

The **human-readable summary** contains the strategy name, the initial and final dates, initial cash, final cash, final holdings, final portfolio value, and total return. You may choose the exact layout. The report does not need to print the individual orders. If holdings of a security are zero, omit them from the report.

For example, the scenario in the **Accounting Reference (https://canvas.cornell.edu/courses/90271/pages/a2-accounting-reference)** could produce this report:

```
Strategy: my-strategy
Period: 2024-01-02 to 2024-01-04
Initial cash: $100.00
Final cash: $63.00
Final holdings:
  ALFA: 2 shares
  BETA: 1 share
Final portfolio value: $106.00
Total return: 6.00%
```

The exact labels and spacing need not match this example.

R6.1: Write the equity CSV to a file as described above.

R6.2: Print the complete human-readable final summary to standard output as described above.

R7. Validation Evidence

Alternative f

Imagine delivering your backtester to a client who wants to rely on its results for investment decisions. Your client would want evidence that the finished program behaves correctly, not merely a promise that it does!

Your evidence will consist of four test groups whose expected results can be determined independently of your implementation. For each of the three OUnit groups, check the expected cash, holdings, and portfolio value on every date.

1. **Cash on the supplied tiny market file.** Run the cash strategy on `data/handout.csv` with `$100.00`. Use OUnit to check the expected results.
2. **Buy-and-hold.** Run buy-and-hold on `data/handout.csv` *and* on a new, tiny market file of your own with multiple dates and securities. Use OUnit to check the expected results. A good new file would use new security names, leave some cash unspent, and change the held security's price on later dates.
3. **Stateful strategy.** Run your stateful strategy on `data/handout.csv` and on a new, tiny market file of your own with multiple dates and securities. (It's fine if that's the same file as in the previous test.) Use OUnit to check the expected results. A good new file would exercise stateful behaviors particular to your strategy.
4. **Command-line reports.** Use Cram to run the program on `data/small-spy-5days.csv` with `$10,000.00` under all three strategies: cash, buy-and-hold, and your stateful strategy. Calculate the expected results, then check the complete standard output (as required in R6.2) from all three runs.

To create additional historical price datasets, you may use the optional **a2-tools.zip (https://canvas.cornell.edu/courses/90271/files/16180858?wrap=1)** **(https://canvas.cornell.edu/courses/90271/files/16180858/download?download_frd=1)** package to convert HSMD files into A2 CSVs. Its README explains setup and use; the package is not needed for the required tests.

Explain your new market files as part of your testing documentation in `docs/PROCESS.md`.

R7.1: Include the three OUnit test groups described above.

R7.2: Include the end-to-end Cram test group over the supplied small market file, checking the complete standard output for all three strategies.

R7.3: In `docs/PROCESS.md`, explain the expected results for the four test groups and how your new market files exercise the strategies.

Rubric

Component	Criterion	Points
Submitted program	Market data input, portfolio accounting, command-line behavior, report, and equity output	15
Submitted program	Required strategy behavior, modular architecture, hidden state, and protection against future-data access	15
Submitted program	Automated tests, independent validation evidence, and the written validation argument	15
Submitted program	Incremental planning, testing, and thoughtful revision	5
Technical interview	Arriving on time and ready to begin for the interview slot you booked	25
Technical interview	Explaining and justifying your code, demonstrating that you understand it, and answering questions about it	15
Technical interview	Making and explaining a requested change to your code during the interview	10
Total		**100**

Technical Interview Question Bank

The same general instructions as A0 apply here. Please be sure to turn off any AI assistants or tools before answering.

1. **Reading market data.** Choose one row from a market file used by your tests. Trace how your program converts its CSV fields into the date, security names, and integer-cent prices used by the simulation. How does your representation preserve the association between each security and its price?

2. **Advancing the simulation.** Trace one date through your backtest engine. What information does the strategy receive, what does it return, when are its orders processed, and when is portfolio value recorded? Show where the strategy state and portfolio are carried forward to the next date.

3. **Processing orders in sequence.** Explain how your implementation processes a sale followed by a purchase that needs the sale proceeds. What could change if those orders were reversed? Then explain how a rejected order leaves the portfolio unchanged while allowing later orders to be considered.

4. **Accounting precisely.** Choose a buy-and-hold example that leaves some cash unspent. Calculate the first purchase, remaining cash, and a later portfolio value, and connect those calculations to your code. Why does portfolio accounting use integer cents, and how do you avoid integer division when calculating percentage return?

5. **Hiding strategy state.** Show your common strategy signature and one strategy's implementation. What is that strategy's state representation, and how is it hidden from clients? Explain how the engine can initialize and carry the state without knowing its representation.

6. **Making stateful decisions.** Choose two consecutive dates from a test of your stateful strategy. Explain the state before each call, the proposed orders, and the state returned afterward. Identify information retained from an earlier date that affects a later decision.

7. **Keeping the engine independent.** Suppose you added a fourth strategy with a different state representation and trading rule. Which parts of your program would need to change, and which could remain unchanged? Show how the existing engine interacts with strategies through their common signature.

8. **Preventing future-data access.** Show what your strategy receives during initialization and on each date. Explain how your interface excludes other price rows and how your engine supplies the correct current row. If a later price row were changed, why should results for earlier dates remain unchanged?

9. **Reporting errors.** Choose an invalid command line or an input filename that cannot be opened. Trace how your program detects the problem and reports it with an unsuccessful exit status. For buy-and-hold, why does checking the selected security require information from the market file?

10. **Justifying an OUnit test.** Choose one required OUnit test on a market file you created. Explain how you determined its expected results independently of your implementation, which dated states it checks, and a plausible defect it would reveal. Why did you choose those prices and securities?

11. **Checking the executable.** Choose one required Cram run. Explain what its command and expected-output lines check, including how the reported final value and return follow from the market data. What does this test check that your OUnit tests alone do not establish, and what does it leave unchecked?

12. **Planning and revising development.** Choose one end-to-end task from `docs/PROCESS.md`. Explain the working behavior you planned, the concrete test you performed, and what its result taught you. Describe a consequential revision and its reason; if none was needed, explain how the evidence supported continuing with your original plan.

Final Checklist

> Before submitting, confirm each of the following:
>
> - From the top-level project directory, `dune build` succeeds without warnings and `dune test` runs to completion.
> - The project follows the **CS 3110 Programming Guidelines (https://canvas.cornell.edu/courses/90271/pages/cs-3110-programming-guidelines)**, including formatting with `ocamlformat`.
> - I have reviewed every numbered requirement in R0–R7 and confirmed that my final submission satisfies all of them.
> - I have run cash, buy-and-hold, and my stateful strategy on a valid market file, and I have checked the complete final summary, equity CSV, and representative command-line errors.
> - My strategy modules share the required interface, keep their state representations and strategy-specific helpers hidden, run through the common engine, and receive prices only from the current price row on each call.
> - My OUnit suite checks cash on the supplied tiny market file and checks buy-and-hold and my stateful strategy on both that file and tiny market files of my own. My Cram suite checks all three strategies' complete summaries on the supplied small market file.
> - `docs/PROCESS.md` is complete, every recorded result and claim matches the code and evidence I am submitting, and my submission includes the market files needed by its tests.
> - My project contains an `AUTHORS.md` file listing every collaborator, or stating that I worked alone.
> - After uploading, I verified that Gradescope received the version I intend to submit.

Alternative f